



Quantum vs Classical Pricing for Column Generation

Per-PSP comparison on $n = 20$ Erdős–Rényi graphs

QCi Research

Quantum Computing Inc

May 12, 2026

Outline

- 1 Problem setup
- 2 Column generation
- 3 Branch-and-Price
- 4 Dirac pricing
- 5 Zoom-in: CG on ER(20, 0.7)
- 6 Summary

Graph coloring

Minimum Vertex Coloring. Assign colors to vertices so that adjacent vertices differ, using as few colors as possible. $\chi(G)$ is the chromatic number.

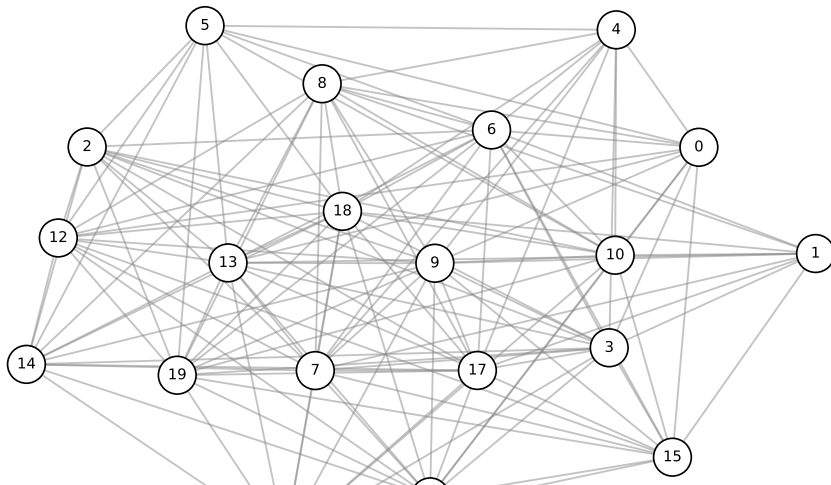
- NP-hard in general; equivalent to partitioning V into independent sets.
- Each color class is an **independent set** (IS) of G .

IS-partition view

Minimum coloring = minimum number of ISs that cover V . This is a **set-cover** ILP whose columns are the ISs of G .

Our running instance: ER(20, 0.7, seed=0)

ER(20, $p=0.7$, seed=0) $|V|=20$ $|E|=131$



Column generation in one page

Master LP (relaxation):

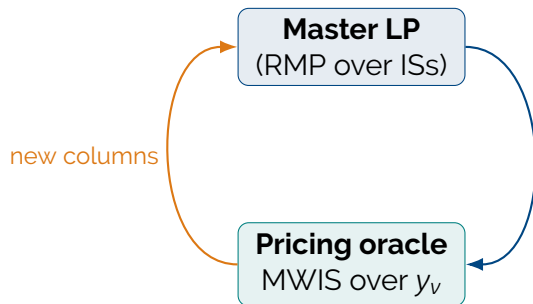
$$\min_{y \geq 0} \sum_{S \in \mathcal{S}} y_S \quad \text{s.t.} \quad \sum_{S \ni v} y_S \geq 1 \quad \forall v.$$

$\mathcal{S}$ = all ISs of G is exponential; start with a small subset and **generate columns as needed**.

Pricing subproblem (**PSP**) at dual y_v :

$$\max_{S \text{ IS}} \sum_{v \in S} y_v - 1 \quad (\text{negative reduced cost})$$

Any S with $\sum_v y_v > 1$ is profitable.



The pricing subproblem is where pricing oracles compete

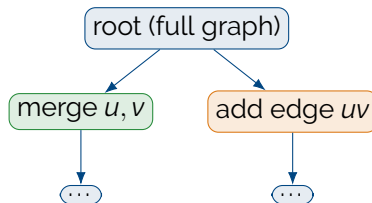
Key observation

On a fixed (G, y) the PSP is NP-hard as MWIS. Any oracle that returns profitable ISs works. **The more columns per call, the fewer master solves needed to close the LP.**

Two oracles, same PSP interface:

- **LP pricing**: solve MWIS LP relaxation with `scipy.linprog`, multi-threshold support extraction + random pruning.
- **Dirac pricing**: submit Motzkin–Straus QP to Dirac-3; 100 raw samples; multi-threshold / multi-prune / randomized rounding post-processing.

Branch-and-Bound tree over colorings



- Each node runs its own CG loop on a *modified* graph.
- Ryan–Foster binary branching: **merge** (same color) or **add edge** (different colors) on a fractional pair.
- The PSP changes as the graph is modified — new subproblems to price.

Two branching rules in this repo

IS-fixing (ISF) single-child path tree — fix one IS per level, shrink the remaining graph.
Original paper method. Cheap but weak lower bound.

Ryan-Foster (RF) binary tree on a fractional pair (u, v) : left merges them, right adds the edge. Monotone LP lower bound; can **prove optimality** when the tree is exhausted.

Motzkin–Straus for MWIS

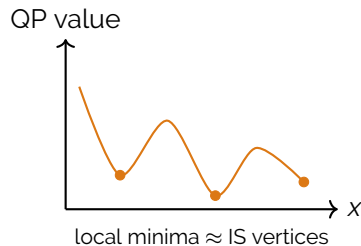
The weighted Motzkin–Straus trick reformulates max-weight IS as a quadratic program on the simplex:

$$\max_{x \geq 0, \mathbf{1}^\top x = 1} \sum_v w_v x_v^2 - \sum_{(u,v) \in E} x_u x_v$$

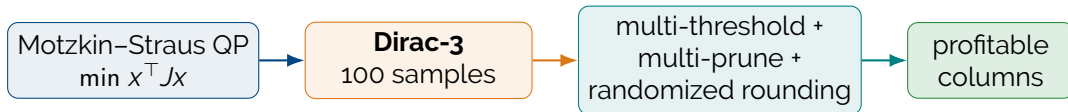
Dirac-3 **directly supports** this primitive:

$\min x^\top C + x^\top Jx$ with $C = 0, J = \text{graph matrix}$.

- Returns `num_samples=100` real vectors of length $|V|$.
- Post-processing extracts profitable ISs from these vectors.



Dirac's raw response, post-processed into columns



- Each sample is a continuous vector of length $|V|$.
- Thresholds $\{0.005, 0.01, 0.05, 0.1, 0.2\}$ extract support sets.
- Pruning strategies + local search turn each support into a valid IS.

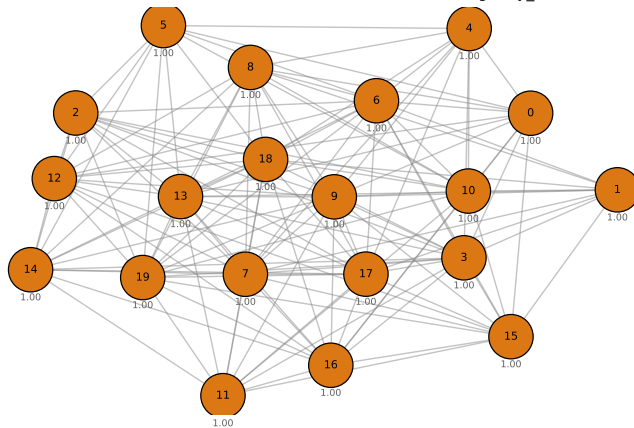
Pick two pricing subproblems from the QCG run

On ER(20, 0.7): QCG runs 6 pricing calls. We compare the **Dirac pricing** vs **LP pricing** outputs on two representative PSPs:

PSP #1	call_00	uniform duals $y_v = 1/\chi$
PSP #2	call_03	differentiated duals (mid CG iter)

PSP #1: dual weights (uniform, first CG iter)

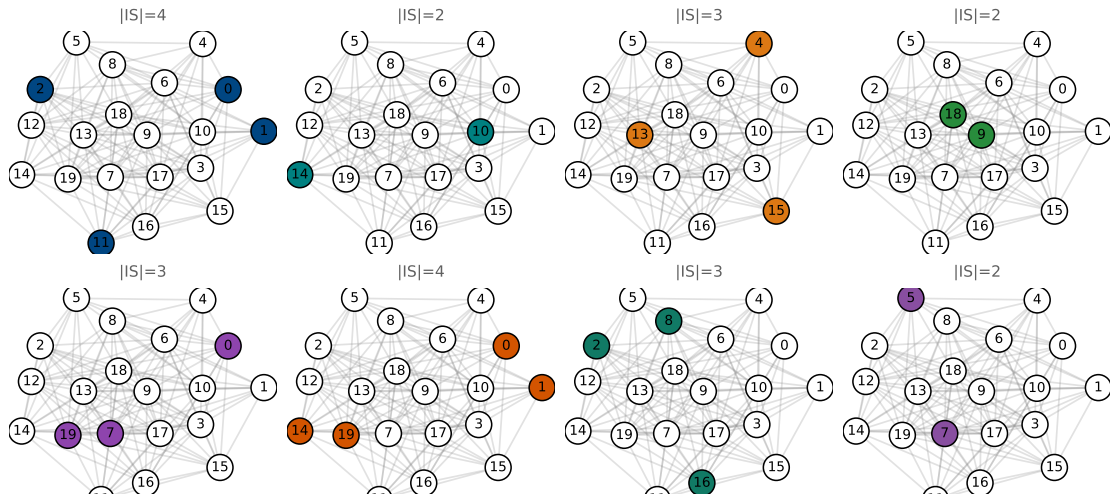
PSP #1: first CG iter (uniform duals) — dual weights y_v



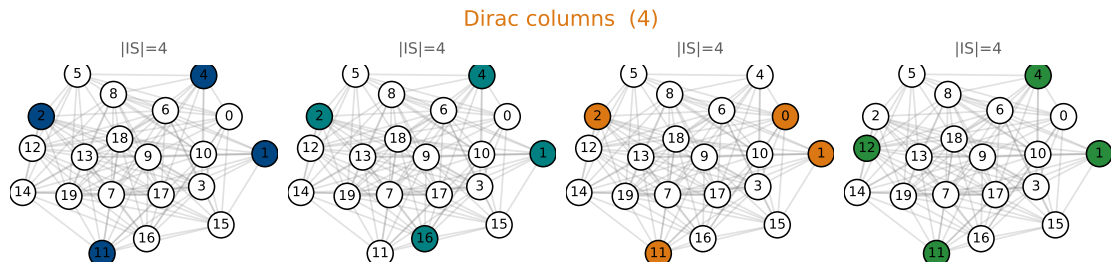
All y_v equal ≈ 0.13 — the PSP is effectively unweighted MWIS.

PSP #1: LP pricing columns (16)

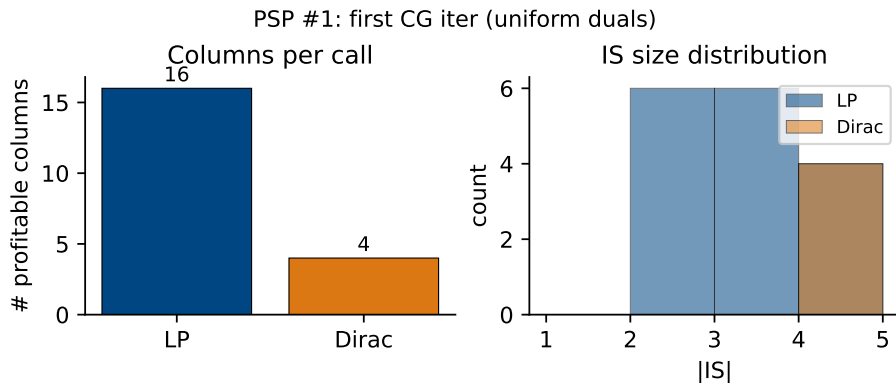
LP columns (16)



PSP #1: Dirac pricing columns (4)



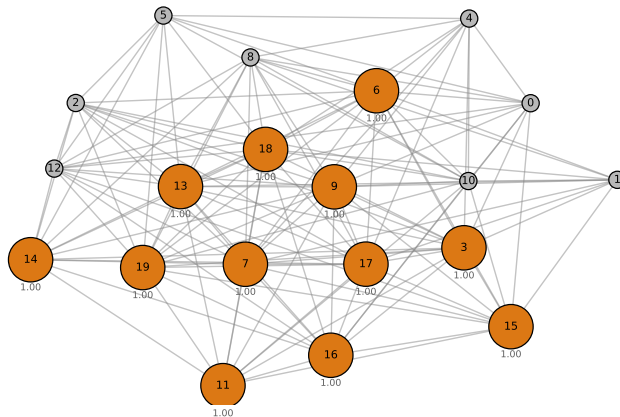
PSP #1: stats



On uniform duals, the LP returns an *integral-ish* polytope vertex from which many support-threshold variants expose distinct maximum ISs. Dirac's samples spread (no direction) and the extractor finds fewer.

PSP #2: dual weights (differentiated, mid CG iter)

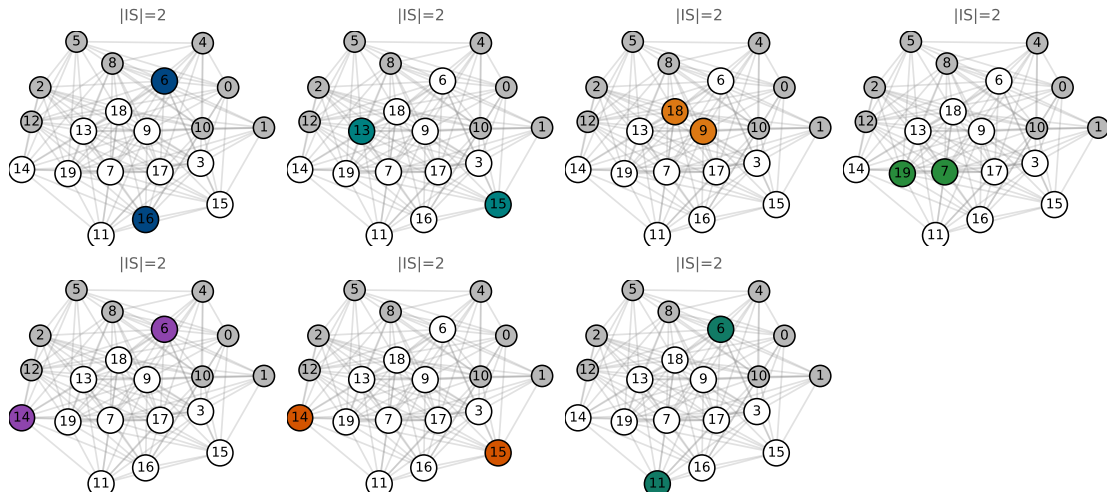
PSP #2: mid CG iter (differentiated duals) — dual weights y_v



After 3 CG iterations the LP concentrates weight on hard-to-cover vertices; 8 vertices have been filtered out (grey).

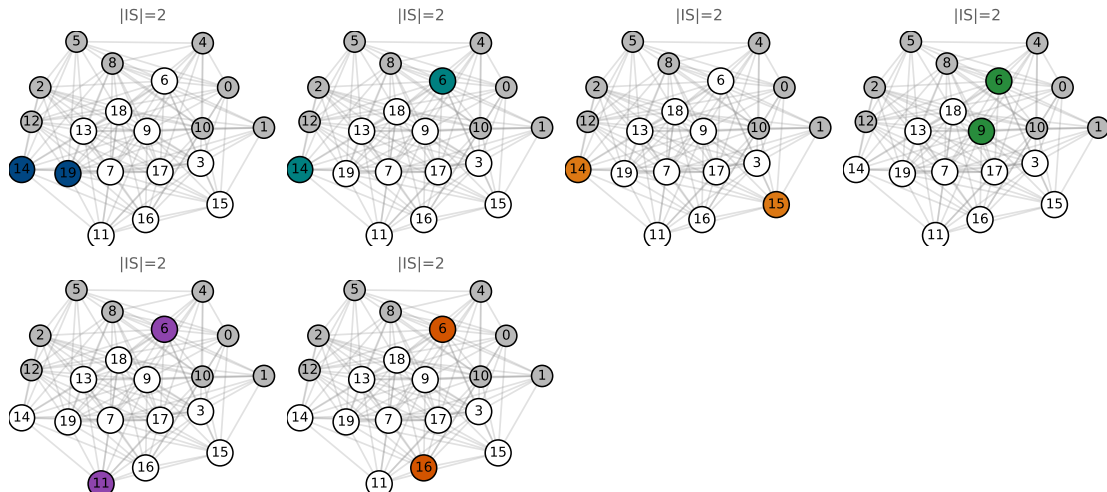
PSP #2: LP pricing columns

LP columns (7)

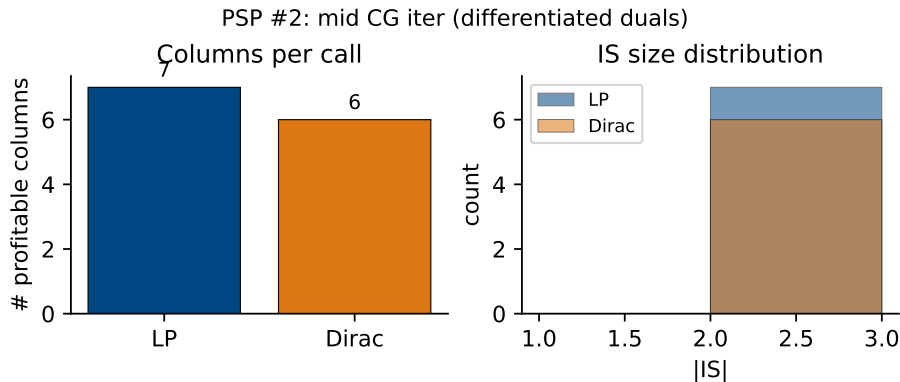


PSP #2: Dirac pricing columns

Dirac columns (6)

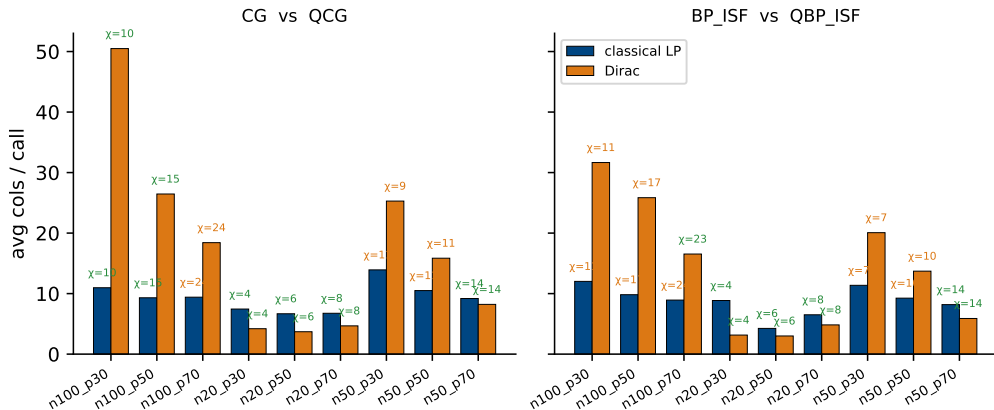


PSP #2: stats



Columns-per-call: aggregate view across the benchmark suite

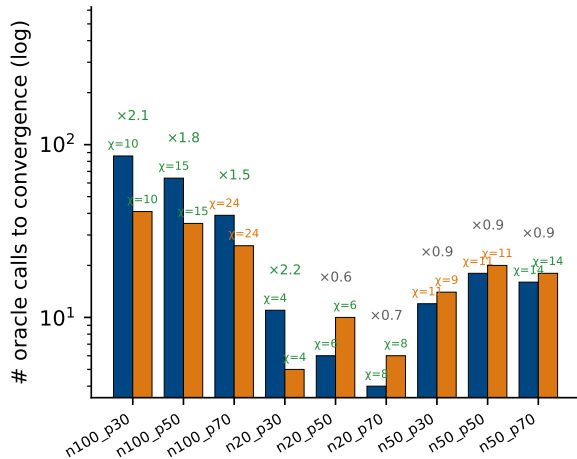
Columns-per-call across all benchmarked instances (χ label above each bar; **green**=matches best-known χ , **orange**=worse)



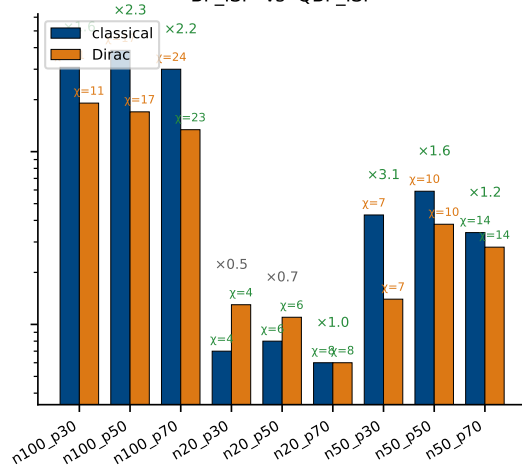
Iterations to solution

Iterations (pricing-oracle calls) to solution — $\times N$ labels show classical/quantum ratio

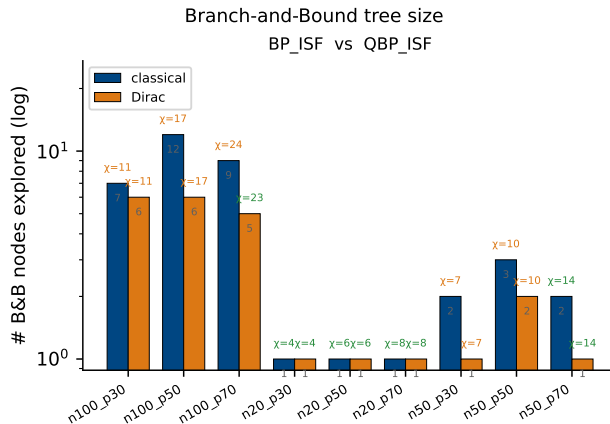
CG vs QCG



BP_ISF vs QBP_ISF



Branch-and-Bound tree size (IS-fixing)



QBP-ISF closes the tree with **fewer or equal B&B nodes** than classical BP-ISF on the $n = 100$ instances, consistent with Dirac's richer column pool per call.

Full benchmark table

Instance	reference		pricing method: χ (iters / cols-per-call)			
	MILP	Hexaly	CG	QCG	BP-ISF	QBP-ISF
$n=20, p=0.3$	4 ✓	4 ✓	4 (11/7.5)	4 (5/4.2)	4 ✓ (7/8.9)	4 ✓ (13/3.1)
$n=20, p=0.5$	6 ✓	6 ✓	6 (6/6.7)	6 (10/3.7)	6 ✓ (8/4.2)	6 ✓ (11/3.0)
$n=20, p=0.7$	8 ✓	8 ✓	8 (4/6.8)	8 (6/4.7)	8 ✓ (6/6.5)	8 ✓ (6/4.8)
$n=50, p=0.3$	6 ✓	6 ✓	11 (12/13.9)	9 (14/25.3)	7 (43/11.4)	7✓ (14/20.1)
$n=50, p=0.5$	9	9	11 (18/10.5)	11 (20/15.8)	10 (59/9.2)	10 (38/13.7)
$n=50, p=0.7$	14	14	14 (16/9.2)	14 (18/8.2)	14 ✓ (34/8.2)	14 ✓ (28/5.9)
$n=100, p=0.3$	10	10	10 (86/11.0)	10 (41/50.5)	11 (308/12.0)	11 (191/31.7)
$n=100, p=0.5$	17	17	15 (64/9.3)	15 (35/26.5)	17 (386/9.8)	17 (170/25.9)
$n=100, p=0.7$	28	23	24 (39/9.4)	24 (26/18.4)	24 (300/8.9)	23 (134/16.5)

bold = matches the best-known χ for that instance; ✓ = method proves optimality. " χ (iters / cpc)" reports chromatic number / oracle calls / avg columns per call. For $n=100$ $p=0.3, 0.5$ the final set-cover ILP initially timed out on the large replayed column pool with HiGHS; re-solving with **Hexaly** (600 s cap) recovers $\chi = 10$ ($p=0.3$) and $\chi = 15$ ($p=0.5$).

Takeaways

- At **small** n ($n=20$) and early CG iterations, **LP pricing** extraction is competitive or better — integral LP vertices expose multiple ISs through simple thresholding.
- At **large** n ($n=100$), **Dirac pricing** wins both **cols-per-call** ($2-4\times$) and **iterations-to-solution** ($1.5-2.3\times$ fewer oracle calls for CG and IS-fixing B&P). Sample diversity from 100 raw solutions dominates LP's deterministic polytope vertex.
- **Per-call latency** on Dirac (~ 50 s device + queue) shifts the advantage into *call count*, not *wall seconds*. For time-to-solution, classical remains faster on these instances.
- All raw Dirac samples and LP primal vectors are persisted for offline post-processing ablations — no new API calls needed.